

Отчёт по лабораторной работе №7

по дисциплине «Архитектура компьютера»

Дембирел Тимур Леонидович

Российский университет дружбы народов
Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей
Москва 2025г.

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
3	Вывод	16

Список иллюстраций

2.1	Создаем каталог и файл	7
2.2	Вводим текст программы	8
2.3	Проверяем	8
2.4	Изменяем текст программы	9
2.5	Создаем и проверяем	9
2.6	Изменяем текст программы	10
2.7	Проверяем	10
2.8	Создаем и вводим	11
2.9	Проверяем	11
2.10	Изображение	12
2.11	Изображение	12
2.12	Изображение	12
2.13	Удаляем один операнд	12
2.14	Транслируем	12
2.15	Изменение в листинге	13
2.16	Программа	14
2.17	Проверяем	14
2.18	Программа	15
2.19	Проверяем	15

Список таблиц

2.1	Основные команды, используемые при работе	7
-----	---	---

1 Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2 Выполнение лабораторной работы

В Таблица 2.1 приведены основные команды, используемые при работе с git, элементами программирования и компиляцией программ.

Таблица 2.1: Основные команды, используемые при работе

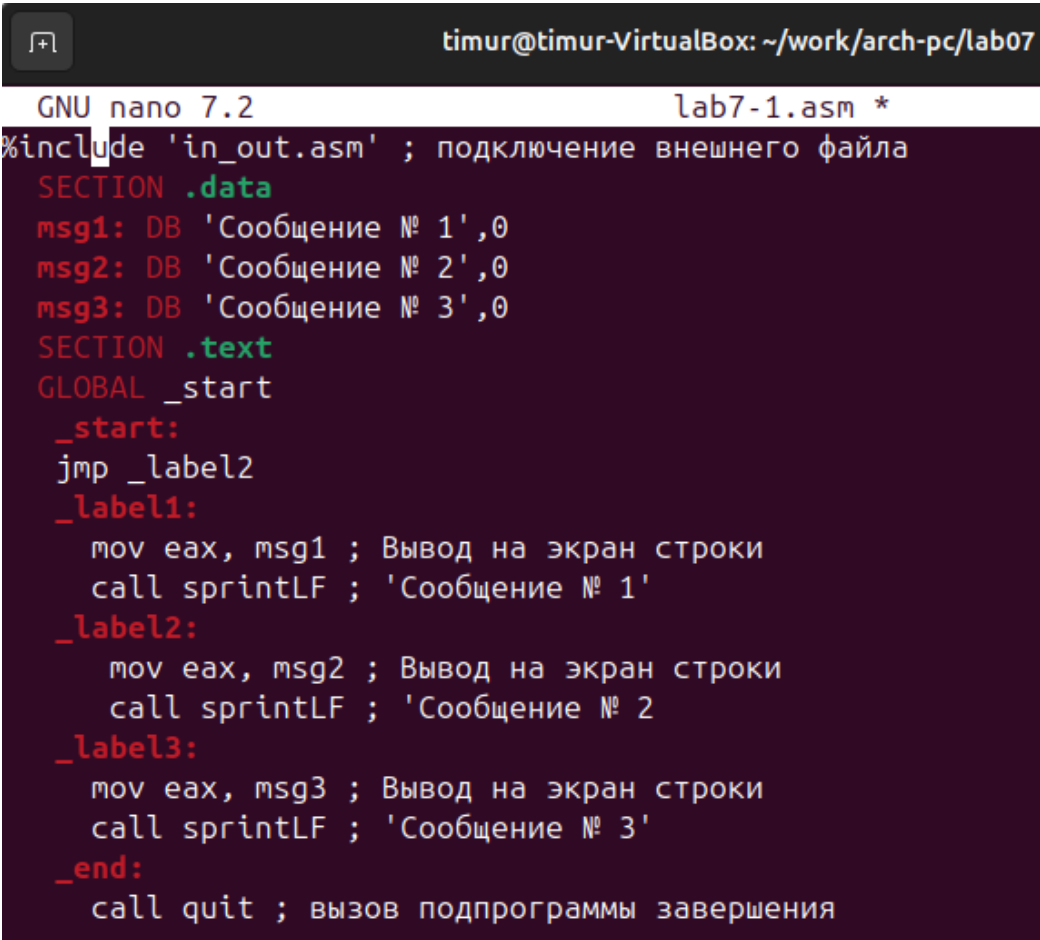
Команды	Описание
<code>touch</code>	Создание файла
<code>nasm</code>	Компилятор ассемблера NASM
<code>git add</code>	Добавление изменений в индекс
<code>git commit</code>	Создание коммита
<code>git push</code>	Отправка изменений на удаленный репозиторий
<code>ld</code>	Связывает объектные файлы в исполняемую программу
<code>mkdir</code>	Создает новую директорию
<code>mcedit</code>	Текстовый редактор в терминале

Создадим каталог для программ лабораторной работы № 7, перейдем в него и создадим файл „lab7-1.asm“ рис. 2.1.

```
timur@timur-VirtualBox:~$ mkdir ~/work/arch-pc/lab07
timur@timur-VirtualBox:~$ cd ~/work/arch-pc/lab07
timur@timur-VirtualBox:~/work/arch-pc/lab07$ touch lab7-1.asm
```

Рисунок 2.1: Создаем каталог и файл

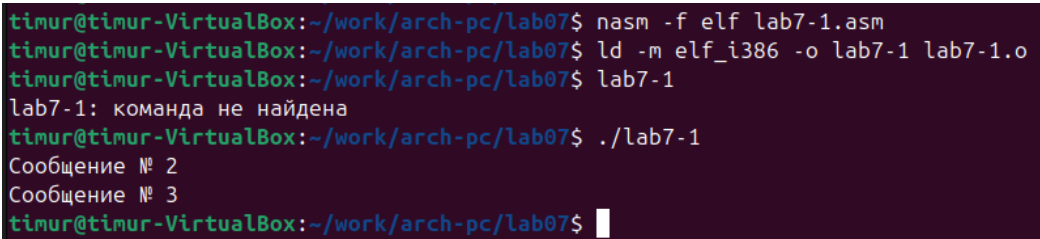
Рассмотрим пример программы с использованием инструкции „jmp“. Введем в файл „lab7-1.asm“ текст программы из листинга 7.1 рис. 2.2.



```
timur@timur-VirtualBox: ~/work/arch-pc/lab07
GNU nano 7.2 lab7-1.asm *
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
    mov eax, msg1 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 1'
_label2:
    mov eax, msg2 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 2'
_label3:
    mov eax, msg3 ; Вывод на экран строки
    call sprintf ; 'Сообщение № 3'
_end:
    call quit ; вызов подпрограммы завершения
```

Рисунок 2.2: Вводим текст программы

Создаем исполняемый файл и запускаем его рис. 2.3.



```
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab07$ lab7-1
lab7-1: команда не найдена
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
timur@timur-VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.3: Проверяем

Изменим текст программы, чтобы он выводил сначала „сообщение №2“, а затем

„сообщение №1“, в соответствии с листингом 7.2 рис. 2.4.

```
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

Рисунок 2.4: Изменяем текст программы

Создаем исполняемый файл и проверяем его работу рис. 2.5.

```
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-1cp.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1cp lab7-1cp.o
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ./lab7-1cp
Сообщение № 2
Сообщение № 1
timur@timur-VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.5: Создаем и проверяем

Изменим текст программы добавив или изменив инструкции „jmp“, чтобы он выводил сообщения с 3 по 1 номер рис. 2.6.

```
timur@timur-VirtualBox: ~/work/arch-pc/lab07
GNU nano 7.2 lab7-1.asm
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label3      ; Первый переход к выводу сообщения №3
_label1:
mov eax, msg1    ; Вывод на экран строки
call sprintLF    ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2    ; Вывод на экран строки
call sprintLF    ; 'Сообщение № 2'
jmp _label1      ; Переход к выводу сообщения №1
_label3:
mov eax, msg3    ; Вывод на экран строки
call sprintLF    ; 'Сообщение № 3'
jmp _label2      ; Переход к выводу сообщения №2
_end:
call quit        ; вызов подпрограммы завершения
```

Рисунок 2.6: Изменяем текст программы

Создаем исполняемый файл и проверяем его рис. 2.7

```
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
timur@timur-VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.7: Проверяем

Создаем файл „lab7-2.asm“ в каталоге „~/work/arch-pc/lab07“. Внимательно изучаем текст программы из листинга 7.3 и введите в „lab7-2.asm“ рис. 2.8.

```

timur@timur-VirtualBox: ~/work/arch-pc/lab07
GNU nano 7.2 lab7-2.asm *
#include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'

```

Рисунок 2.8: Создаем и вводим

Далее создаем исполняемый файл и проверяем его рис. 2.9.

```

timur@timur-VirtualBox:~/work/arch-pc/lab07$ touch lab7-2.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nano lab7-2.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm

timur@timur-VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o

timur@timur-VirtualBox:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 64
Наибольшее число: 64
timur@timur-VirtualBox:~/work/arch-pc/lab07$

```

Рисунок 2.9: Проверяем

Создаем файл листинга для программы „lab7-2.asm“, откроем его с помощью „mcedit“. Внимательно ознакомимся с его форматом и содержанием. Подробно объ-

ясним три строки листинга по выбору: 1)Строка 1 „%include 'in_out.asm“ „. Подключает внешний файл 'in_out.asm“, который содержит определения функций ввода-вывода (таких как „sprintf“, „slen“, „quit“ и др.) рис. 2.10.

```
1 %include 'in_out.asm'
```

Рисунок 2.10: Изображение

2)Строка 8 „8 00000003 803800 <1> cmp byte [eax], 0“. Сравнивает байт по адресу в регистре EAX с нулём рис. 2.11.

```
8 00000003 803800 <1> cmp byte [eax], 0..
```

Рисунок 2.11: Изображение

3)Строка 14 „0000000B 29D8 <1> sub eax, ebx“. Вычитает значение регистра EBX из EAX (eax = eax - ebx) рис. 2.12.

```
14 0000000B 29D8 <1> sub eax, ebx
```

Рисунок 2.12: Изображение

Откроем файл с программой „lab7-2.asm“ и в любой инструкции с двумя операндами удаляем один операнд рис. 2.13.

```
mov ecx,[A] ; 'ecx = A'
mov [max]
```

Рисунок 2.13: Удаляем один операнд

Выполняем трансляцию с получением файла листинга рис. 2.14.

```
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
lab7-2.asm:26: error: invalid combination of opcode and operands
timur@timur-VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.14: Транслируем

Вышла ошибка „invalid combination of opcode and operands“, такая же ошибка появляется и в листинге рис. 2.15.

```
25 00000110 8B0D[35000000]      mov ecx,[A] ; 'ecx = A'
26                               mov [max]
26      *****
26                               error: invalid combination of opcode and operands
```

Рисунок 2.15: Изменение в листинге

В результате этих изменений создается только листинг „lab7-2.lst“, а файл трансляции „lab7-2.o“ не создается и прерывается с ошибкой.

#Задание для самостоятельной работы

1. Напишите программу нахождения наименьшей из 3 целочисленных переменных a, b и c. Значения переменных выбрать из табл. 7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу рис. 2.16 рис. 2.17.

```

GNU nano 7.2                                lab7-task1.asm *
#include 'in_out.asm'

SECTION .data
    msg1 db 'Наименьшее число: ',0h
    A dd 94      ; храним как числа, а не символы
    B dd 5
    C dd 58
SECTION .bss
    min resb 10
SECTION .text
GLOBAL _start
_start:
    ; --- Записываем 'A' в переменную 'min'
    mov ecx,[A] ; 'ecx = A'
    mov [min],ecx ; 'min = A'
    ; --- Сравниваем 'A' и 'B'
    mov ecx,[min]
    cmp ecx,[B] ; Сравниваем min и B
    jl check_C ; Если min < B, переходим к сравнению с C
    mov ecx,[B] ; Иначе min = B
    mov [min],ecx
check_C:
    ; --- Сравниваем текущий минимум с 'C'
    mov ecx,[min]
    cmp ecx,[C] ; Сравниваем min и C
    jl _end ; Если min < C, переходим к выводу
    mov ecx,[C] ; Иначе min = C
    mov [min],ecx

```

Рисунок 2.16: Программа

```

timur@timur-VirtualBox:~/work/arch-pc/lab07$ nano lab7-task1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-task1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-task1 lab7-task1.o
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ./lab7-task1
Наименьшее число: 5
timur@timur-VirtualBox:~/work/arch-pc/lab07$

```

Рисунок 2.17: Проверяем

2. Напишите программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрать из таблицы 7.6 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу для значений x и a из

7.6 рис. 2.18 рис. 2.19.

```
GNU nano 7.2 lab7-task2.asm
%include 'in_out.asm'

SECTION .data
    msg_x db 'Введите x: ',0h
    msg_a db 'Введите a: ',0h
    msg2 db "Результат f(x): ",0h

SECTION .bss
    x resb 10
    a resb 10

SECTION .text
GLOBAL _start
_start:
    ; --- Вывод сообщения 'Введите x: '
    mov eax,msg_x
    call sprint
    ; --- Ввод 'x'
    mov ecx,x
    mov edx,10
    call sread
    ; --- Преобразование 'x' из символа в число
    mov eax,x
    call atoi
    mov [x],eax
    ; --- Вывод сообщения 'Введите a: '
    mov eax,msg_a
    call sprint
```

Рисунок 2.18: Программа

```
timur@timur-VirtualBox:~/work/arch-pc/lab07$ nasm -f elf lab7-task2.asm
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-task2 lab7-task2.o
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ./lab7-task2
Введите x: 3
Введите a: 4
Результат f(x): 9
timur@timur-VirtualBox:~/work/arch-pc/lab07$ ./lab7-task2
Введите x: 1
Введите a: 4
Результат f(x): 5
timur@timur-VirtualBox:~/work/arch-pc/lab07$
```

Рисунок 2.19: Проверяем

3 Вывод

В ходе данной лабораторной работы я изучил команды условного и безусловного переходов, приобрел навыки написания программ с использованием переходов и познакомился с назначением и структурой файла листинга.